

Конструирование компиляторов

Ответы на вопросы к экзамену
по материалам лекций

1. Компиляторы и интерпретаторы: основные определения

Компилятор. (compiler) из языка S в язык T — это программа, осуществляющая перевод программ с языка S на язык T .

При этом различают три языка:

- S — **исходный язык** (source language) — язык, с которого переводят;
- T — **целевой язык** (target / object language) — язык, на который переводят;
- H — **язык реализации** (implementation / host language) — язык, на котором написан сам компилятор.

Запись $S \xrightarrow{H} T$ обозначает компилятор из S в T , написанный на H .

Транслятор. (translator) — синоним компилятора.

Интерпретатор. (interpreter) для языка L — это программа, осуществляющая *выполнение* программ, написанных на языке L (без предварительного перевода в другой язык).

Компилятор и интерпретатор близки по структуре. Компилятор из A в B можно рассматривать как интерпретатор языка A' , синтаксис которого совпадает с синтаксисом A , а смысл программы p заключается в порождении программы на языке B , вычисляющей ту же функцию, что и p , понимаемая как программа на A .

2. Т-диаграммы

Т-диаграммы — это графическая нотация, позволяющая визуализировать взаимодействия программ, компиляторов и интерпретаторов. Базовые элементы:

- **Программа** p на языке L изображается прямоугольником с подписью языка снизу.
- **Машина**, выполняющая язык M , изображается «уголком» (буквой М-образной формы) с подписью M .
- **Интерпретатор** языка L , написанный на M , изображается фигурой, где L сверху, M снизу.
- **Компилятор** $S \xrightarrow{M} T$ изображается Т-образной фигурой с тремя подписями: S слева (вход), T справа (выход), M снизу (язык реализации).

Правила «стыковки»:

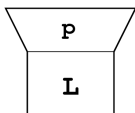
- Чтобы *выполнить* программу на машине, язык программы должен совпадать с языком машины (фигуры стыкуются по языку M).
- **Виртуальная машина.** для языка L — это сочетание интерпретатора языка L , написанного на M , и машины, выполняющей M . Классический

пример — р-машина (виртуальная р-машина Паскаля).

- Чтобы *запустить* компилятор $S \xrightarrow{M} T$, нужна машина для языка M ; на вход подаётся программа на S , на выходе — эквивалентная программа на T .

Базовые фигуры (программа p/L — прямоугольник, язык снизу; машина M — «уголок»):

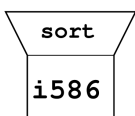
Программа p на языке L :



Машина, выполняющая язык M :



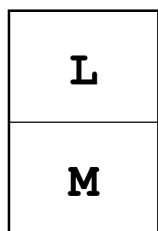
Пример:



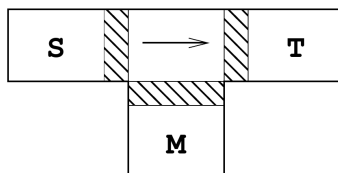
Пример:



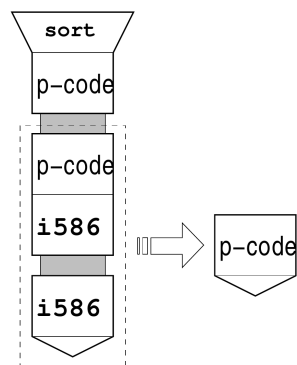
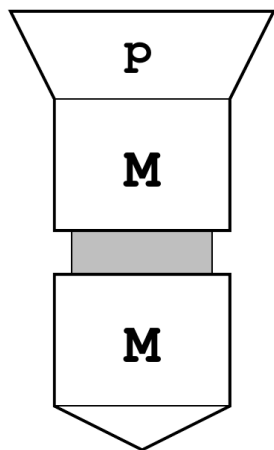
Интерпретатор и компилятор. Интерпретатор языка L , написанный на M , — две полки (L сверху, M снизу). Компилятор $S \xrightarrow[M]{M} T$ — Т-образная фигура: S слева (вход), T справа (выход), M снизу (язык реализации).



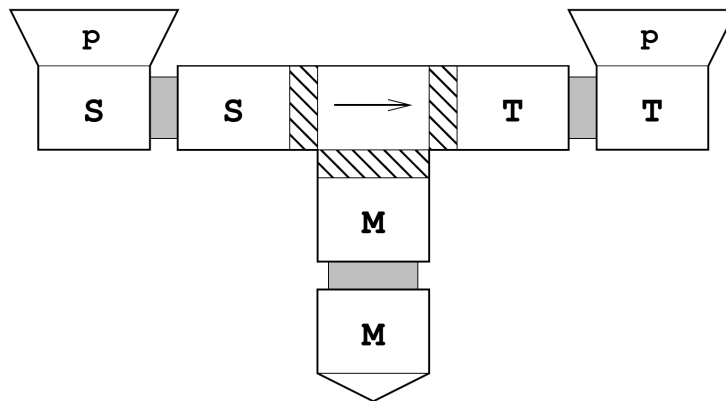
Компилятор $S \xrightarrow[M]{M} T$:



Стыковки. Фигуры соединяются по совпадающим языкам. *Выполнение* программы на машине (языки совпадают), *виртуальная машина* (программа на $L \rightarrow$ интерпретатор $L/M \rightarrow$ машина M):



Работа компилятора: программа p/S подаётся на вход $S \rightarrow T$, на выходе — эквивалентная p/T :



Т-диаграммы особенно удобны для описания процессов раскрутки и переноса самоприменимых компиляторов (см. вопрос 3).

3. Самоприменимые компиляторы: раскрутка и перенос

Самоприменимый компилятор. (self-applicable) — это компилятор вида $S \xrightarrow{S} T$, то есть написанный на том же языке S , который он компилирует.

Если для языка S не существует другого компилятора или интерпретатора, то запуск такого компилятора напрямую невозможен — нужна раскрутка.

Раскрутка. (bootstrapping) — итерационный процесс создания самоприменимого компилятора $S \xrightarrow{S} T$ в случае, когда программы на T запускать можно, а на S — нельзя.

На каждом шаге раскрутки получается компилятор $S_i \xrightarrow{T} T$ для всё большего подмножества языка: $S_1 \subset \dots \subset S_{n-1} \subset S_n \subset S$. Сначала компилятор для маленького подмножества S_1 пишут вручную на T , затем им компилируют компилятор для S_2 , написанный уже на S_1 , и так далее.

Раскрученный самоприменимый компилятор. языка S на платформе T — это пара компиляторов $(S \xrightarrow{S} T, S \xrightarrow{T} T)$ такая, что применение второго (исполнимого) к первому (исходному тексту на S) даёт $S \xrightarrow{T} T$, то есть пара «воспроизводит сама себя».

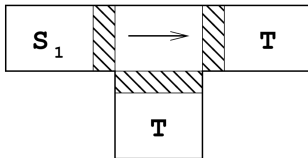
Перенос (порт) на новую платформу T_2 . Имея раскрученный компилятор $(S \xrightarrow{S} T_1, S \xrightarrow{T_1} T_1)$, на основе исходного текста $S \xrightarrow{S} T_1$ разрабатывают $S \xrightarrow{S} T_2$ и выполняют два шага компиляции:

1. имеющимся работающим компилятором $S \xrightarrow{T_1} T_1$ компилируют $S \xrightarrow{S} T_2$ и получают *кросскомпилятор* $S \xrightarrow{T_1} T_2$ (он выполняется на старой платформе, но порождает код для новой);
2. кросскомпилятором компилируют тот же исходник $S \xrightarrow{S} T_2$ и получают $S \xrightarrow{T_2} T_2$ — компилятор, работающий уже на новой платформе.

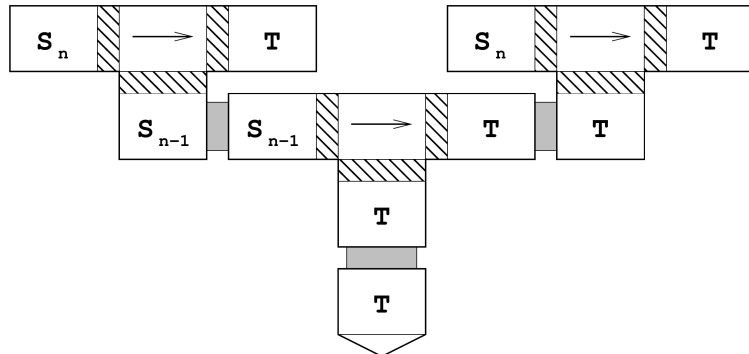
Кросскомпилятор. — компилятор $S \xrightarrow{T_1} T_2$, исполняющийся на одной платформе (T_1), но порождающий код для другой (T_2).

Шаги раскрутки на T-диаграммах. На шаге 1 вручную написанным на T компилятором $S_1 \rightarrow T$ обрабатывают исходники; на шаге n имеющимся компилятором $S_{n-1} \rightarrow T$ (на платформе T) компилируют исходный текст $S_n \rightarrow T$, написанный на подмножестве S_{n-1} , и получают исполнимый $S_n \rightarrow T$ для большего подмножества ($S_1 \subset \dots \subset S_n \subset S$):

Шаг 1.

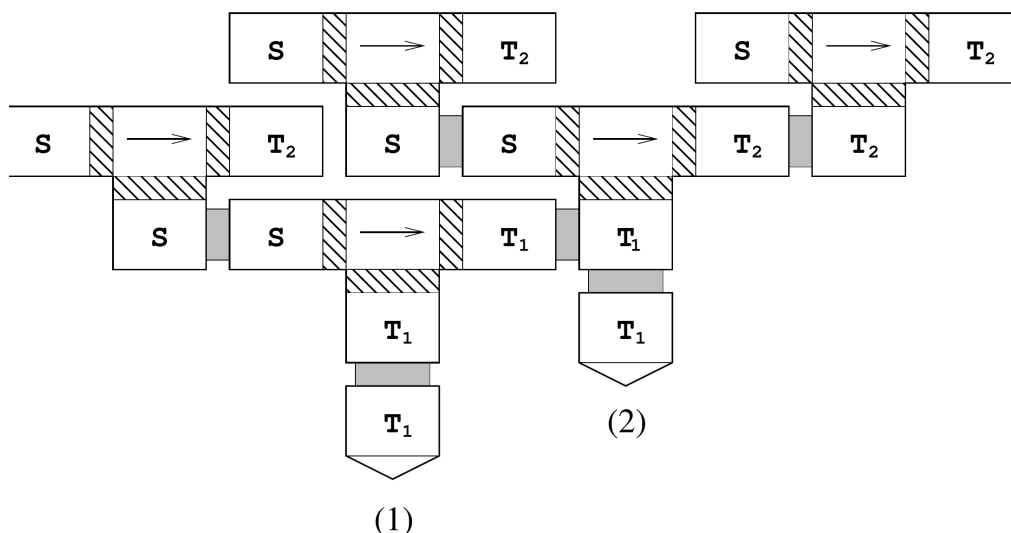


Шаг n.



$$S_1 \subset \dots \subset S_{n-1} \subset S_n \subset S$$

Перенос на новую платформу T_2 . Два шага: (1) старым компилятором $S \xrightarrow{T_1} T_1$ компилируют исходник $S \xrightarrow{S} T_2$ → получают кросскомпилятор $S \xrightarrow{T_1} T_2$; (2) кросскомпилятором компилируют тот же исходник → получают $S \xrightarrow{T_2} T_2$:



4. Общая схема компиляции. Компоновка. Библиотека поддержки времени выполнения

Компилятор преобразует исходную программу в объектную, проходя через промежуточное представление. Выделяют три **стадии**:

- **Анализ** (front-end) — не зависит от целевого языка; на этой стадии порождаются сообщения об ошибках. Если компиляция продолжается после обнаружения ошибки, говорят о *восстановлении при ошибках* (error recovery).

- **Оптимизация** — зависит только от языка промежуточного представления.
- **Синтез** (back-end) — почти не зависит от исходного языка.

Компоновка (компоновка/линковка). Объектные модули, полученные при отдельной компиляции, объединяются компоновщиком в исполняемый файл; при этом разрешаются внешние ссылки между модулями.

Библиотека поддержки времени выполнения (run-time support library) — набор подпрограмм, необходимых программе во время выполнения (управление памятью, ввод-вывод, поддержка исключений, диспетчеризация и т.п.). Код, генерируемый компилятором, опирается на эту библиотеку.

5. Фазы анализа

Стадия анализа — это композиция более простых преобразований, называемых **фазами компиляции**. Фазы анализа выполняются последовательно:

1. **Чтение входного потока** (character handling) — преобразование образа текста в последовательность *кодовых точек* во внутреннем стандарте кодирования (например, UTF-32).
2. **Лексический анализ** (scanning, линейный анализ) — объединение кодовых точек в *лексемы* и порождение токенов (LPAREN, IDENT, PLUS и т.д.).
3. **Синтаксический анализ** (parsing, иерархический анализ) — группировка токенов в синтаксические структуры (дерево разбора).
4. **Семантический анализ** — проверка дерева на соответствие контекстным ограничениям (правила видимости, типы); накопление информации о типах.
5. **Генерация промежуточного представления** — перевод синтаксического дерева в форму, удобную для оптимизации и генерации кода.

Семантический анализ. — фаза, выполняющая проверку синтаксического дерева на соответствие его компонентов *контекстным ограничениям* (видимость идентификаторов, проверка типов и т.п.).

6. Группировка фаз компиляции

Фазы могут работать последовательно или (псевдо)параллельно, образуя сложные сочетания.

Проход. (pass) — это выполнение группы параллельно работающих фаз. Проход может быть реализован отдельной программой.

Параллелизм фаз в рамках прохода обычно означает работу в режиме *сопрограмм*, обменивающихся данными (реального параллелизма нет — фазы выполняются поочерёдно). Если компиляция состоит из последовательного выполнения n проходов, её называют **n -проходной**.

Преимущества однопроходной компиляции:

- нет больших промежуточных структур данных в памяти и временных файлов;
- высокая скорость;
- сообщения об ошибках порождаются в правильном порядке.

Недостатки однопроходной компиляции:

- трудность организации (псевдо)параллельного выполнения фаз; необходимость *обратных поправок* (back-patching) при генерации переходов;
- негативное влияние на исходный язык (требование опережающих объявлений и т.п.);
- невозможность глобальной оптимизации программы.

7. Постановка задачи лексического анализа

Выделение лексической структуры из грамматики языка — проверенная временем декомпозиция задачи компиляции (одну сложную грамматику делят на регулярную лексическую часть и КС-синтаксис).

Лексический домен. — заданное некоторым формальным способом множество строк (часто это регулярный язык, задаваемый регулярным выражением или грамматикой).

Лексема. (lexem) — фрагмент текста исходной программы, принадлежащий некоторому лексическому домену.

Лексическая структура языка. — множество лексических доменов, на котором задано отношение строгого линейного порядка $\prec$. Домен D_1 имеет более высокий приоритет, чем D_2 , если $D_1 \succ D_2$.

Приоритеты нужны для разрешения конфликтов: если один и тот же участок принадлежит нескольким доменам, выбирается домен с наивысшим приоритетом. *Пример:* строка `begin` в Pascal принадлежит и домену ключевых слов, и домену идентификаторов, но домен ключевых слов приоритетнее — поэтому `begin` распознаётся как ключевое слово.

Токен. (token) — описатель лексемы, кортеж $\langle domain, coords, attr \rangle$: лексический домен, координаты лексемы в тексте и атрибут. Атрибут вычисляется функцией $f: D \rightarrow A$. В простейшем случае $A = D$ и f тождественна (атрибутами служат сами лексемы).

Задача лексического анализа. *Дано:* лексическая структура $\langle L, \prec \rangle$ и непустой суффикс p исходной программы (полученный отщеплением уже распознанных лексем). *Требуется:* найти самый длинный непустой префикс l строки p , принадлежащий хотя бы одному домену из L , и построить токен $\langle D, c, a \rangle$, где:

1. D — максимальный по $\prec$ элемент множества $L' = \{d \in L \mid l \in d\}$;
2. c — координаты фрагмента l ;
3. $a = f(l)$ — атрибут.

Ключевой принцип — правило *самого длинного совпадения* (maximal munch).

8. Проектирование объектно-ориентированного лексического анализатора

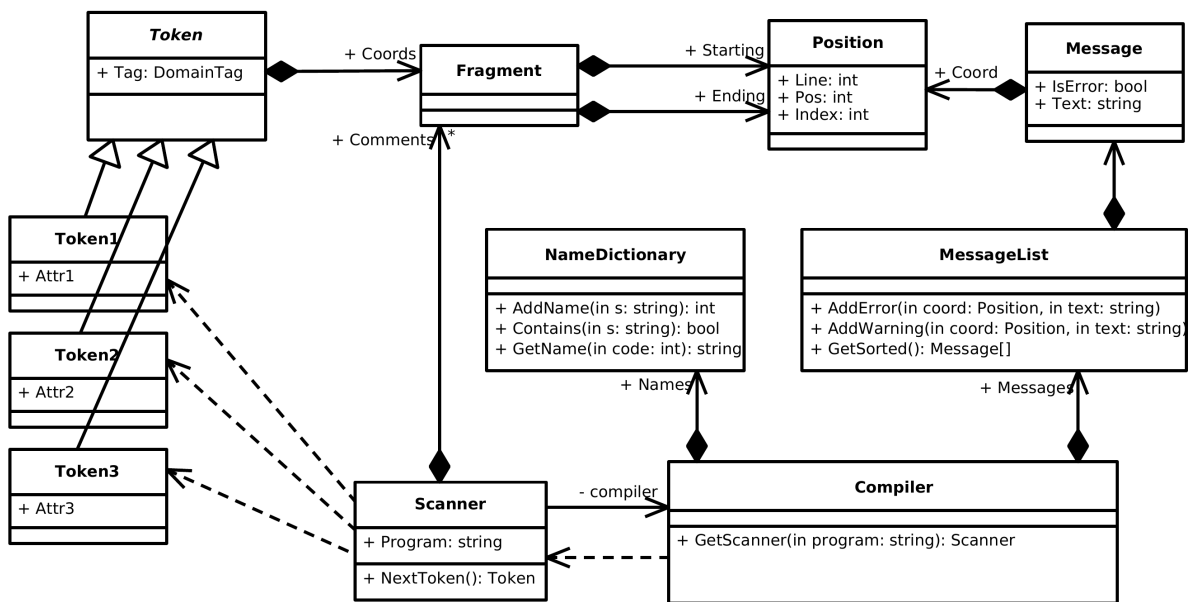
Структуру анализатора удобно описать набором классов:

- **Position** — координаты кодовой точки: Line, Pos, Index (номер строки, позиция в строке, индекс в тексте). Нумерация Line и Pos начинается с 1.

- **Fragment** — координаты фрагмента: пара $\langle \text{Starting}, \text{Ending} \rangle$ из двух `Position`.
- **Message** — сообщение компилятора: `IsError`, `Text`, координата `Coord`.
- **Token** — абстрактный класс токена: координаты `Coords` (`Fragment`) и `Tag` : `DomainTag`. От него наследуются `Token1`, `Token2`, ... (по одному на домен), добавляя свои атрибуты `Attri`. Поле `Tag` нужно, если классы языка не являются самоописывающими типами.
- **DomainTag** — перечисление, в котором каждому домену соответствует своя константа.
- **Scanner** — итератор по токенам; метод `NextToken()` выполняет лексический анализ и попутно ведёт список комментариев. Хранит ссылку на `Compiler`.
- **Compiler** — содержит изменяемые таблицы: `MessageList` (методы `AddError`, `AddWarning`, `GetSorted`) и `NameDictionary` (методы `AddName`, `Contains`, `GetName`). Они заполняются внутри `NextToken`.

Такое разделение отделяет вычисление координат, хранение сообщений и словарь имён от логики распознавания.

Сводная диаграмма классов. `Token` (с `Tag:DomainTag`) — базовый класс, от него наследуют `Token1/2/3`; координаты задаёт `Fragment` из двух `Position`; `Scanner` (`NextToken`) и `Compiler` (с `NameDictionary` и `MessageList`) связаны ссылками:



9. Реализация объектно-ориентированного лексического анализатора вручную

Реализация ведётся на примере конкретной лексической структуры (идентификаторы, числа, символьные литералы, комментарии `/*...*/`, спецсимволы). Основные приёмы:

Position как итератор по тексту. Чтобы не «размазывать» трудоёмкое вычисление координат по коду, объект `Position` делают итератором. Доступ к текущей литере — через свойство `Cp`; переход к следующей — через `operator++`.

```
Position p = new Position("qwerty");
Console.WriteLine(p.Cp); // 'q'
```

```
p++;
Console.WriteLine(p.Cp); // 'w'
```

Внутренняя кодировка — UTF-32, свойство `Cp` понимает суррогатные пары (`Char.ConvertToUtf32`). Предусмотрены предикаты `IsWhiteSpace`, `IsLetter`, `IsLetterOrDigit`, `IsDecimalDigit`.

Рекомендация — маркер конца текста. В конец программы дописывают специальную кодовую точку `-1` (`0xFFFFFFFF`), которой не может быть в тексте. Это упрощает циклы: вместо

```
while (cur.Index < program.Length && cur.Cp == ' ') cur++;
```

пишут просто `while (cur.Cp == ' ') cur++;`

Логика `NextToken` — серия проверок текущей литеры: пропуск пробелов и комментариев, распознавание идентификатора (буква, затем буквы/цифры), числа (с проверкой переполнения через `checked` и контролем разделителя после числа), символьной константы и спецсимволов. Ошибки кладутся в `MessageList`, координаты вычисляются автоматически.

Рекомендация: учитывать ограничения на «примыкание» лексем (например, число должно отделяться от идентификатора разделителем — иначе ошибка `delimiter required`).

10. Лексические распознаватели

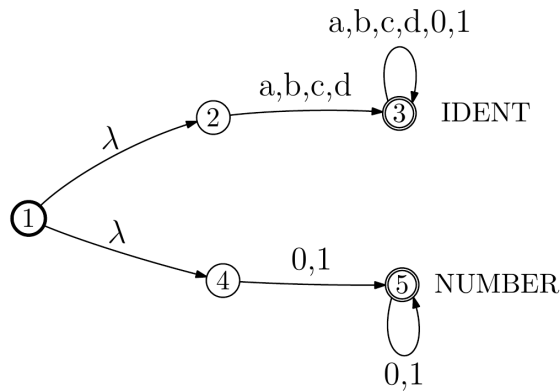
Лексический распознаватель. для языка с алфавитом V и лексической структурой $\langle L, \prec \rangle$ — это конечный автомат $\langle V, Q, q_0, F, \varphi, \delta \rangle$, где: Q — множество состояний; $q_0 \in Q$ — начальное; $F \subseteq Q$ — заключительные состояния; $\varphi: F \rightarrow L$ — функция разметки заключительных состояний (привязывает состояние к домену); $\delta: Q \times (V \cup \{\lambda\}) \rightarrow 2^Q$ — функция переходов (в общем случае недетерминированная, с λ -переходами).

Отличие от обычного конечного автомата — наличие функции разметки φ : распознаватель не просто принимает/отвергает строку, а сообщает, какому домену она принадлежит.

Построение распознавателя по регулярным выражениям (три этапа):

1. для каждого лексического домена строят конечный автомат $N(R)$ по его регулярному выражению (стандартная конструкция Томпсона: для λ , для литеры a , для $X \mid Y$, конкатенации XY , итерации X^* и т.д. — с добавлением λ -переходов);
2. превращают автоматы в распознаватели, добавляя функцию разметки: единственное заключительное состояние привязывают к соответствующему домену;
3. объединяют все распознаватели в один: добавляют новое начальное состояние i и λ -переходы из него к начальным состояниям $i_1, \dots, i_n$ всех частных распознавателей.

Пример. Для $V = \{a, b, c, d, 0, 1\}$, $L = \{\text{IDENT}, \text{NUMBER}\}$, где $\text{IDENT} ::= \text{LETTER} (\text{LETTER} \mid \text{DIGIT})^*$, $\text{NUMBER} ::= \text{DIGIT} \text{ DIGIT}^*$, распознаватель (с λ -переходами из общего начального состояния 1) имеет вид:



11. Детерминизация лексического распознавателя

Детерминированный распознаватель. — распознаватель без λ -дуг, в котором из любого состояния по любому входному символу возможен переход ровно в одно состояние: $\forall q \in Q \forall a \in V (|\delta(q, a)| = 1)$.

Применяется классический алгоритм детерминизации конечных автоматов (subset construction), модифицированный для корректного вычисления функции разметки. Состояния ДКА — это *подмножества* множества состояний НКА.

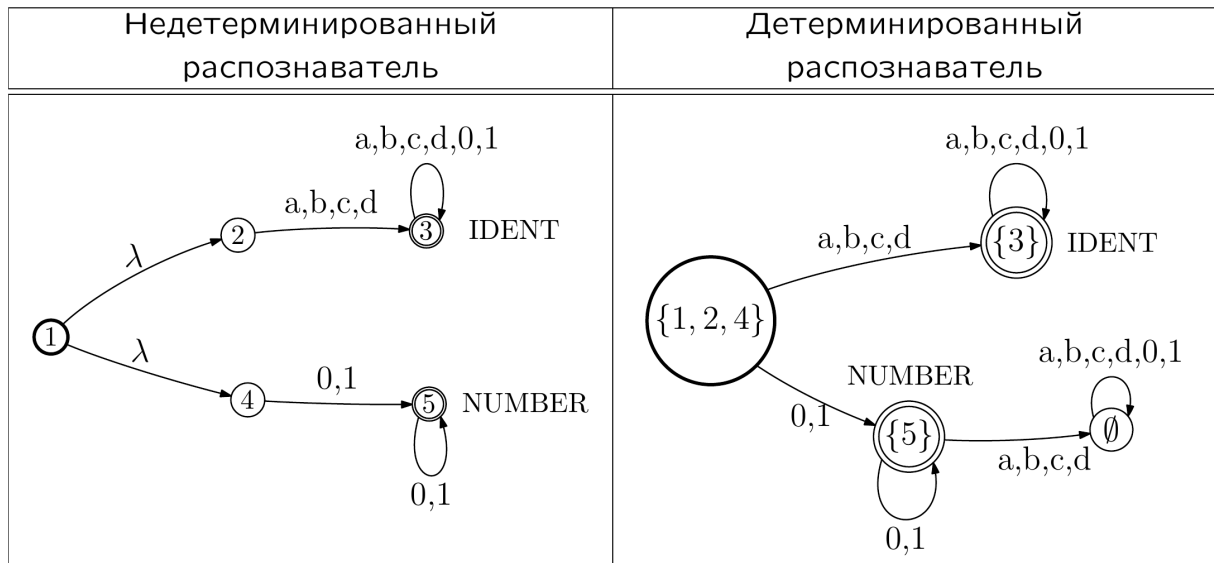
Используется операция λ -closure: $2^Q \rightarrow 2^Q$ — множество состояний, достижимых из заданного множества T по λ -переходам (вычисляется обходом в глубину/ширину со стеком).

Алгоритм Det:

1. начальное состояние ДКА: $q'_0 = \lambda\text{-closure}(\{q_0\})$;
2. для каждого нового состояния T и каждого символа $a \in V$ вычисляют $U = \lambda\text{-closure}(\bigcup_{q \in T} \delta(q, a))$ и полагают $\delta'(T, a) = U$;
3. если U новое — добавляют его в очередь обработки;
4. если $U \cap F \neq \emptyset$, то U — заключительное, и его разметка $\varphi'(U) = \max\{\varphi(q) \mid q \in U \cap F\}$ — выбирается домен с *наивысшим приоритетом* среди достижимых (этим реализуется $\prec$).

Максимизация по φ — ключевая модификация, отличающая детерминизацию распознавателя от детерминизации обычного автомата.

Пример (детерминизация распознавателя из вопроса 10). Состояния ДКА — подмножества состояний НКА; $\{1, 2, 4\} = \lambda\text{-closure}(\{1\})$. Пустое множество $\emptyset$ — состояние-ловушка:



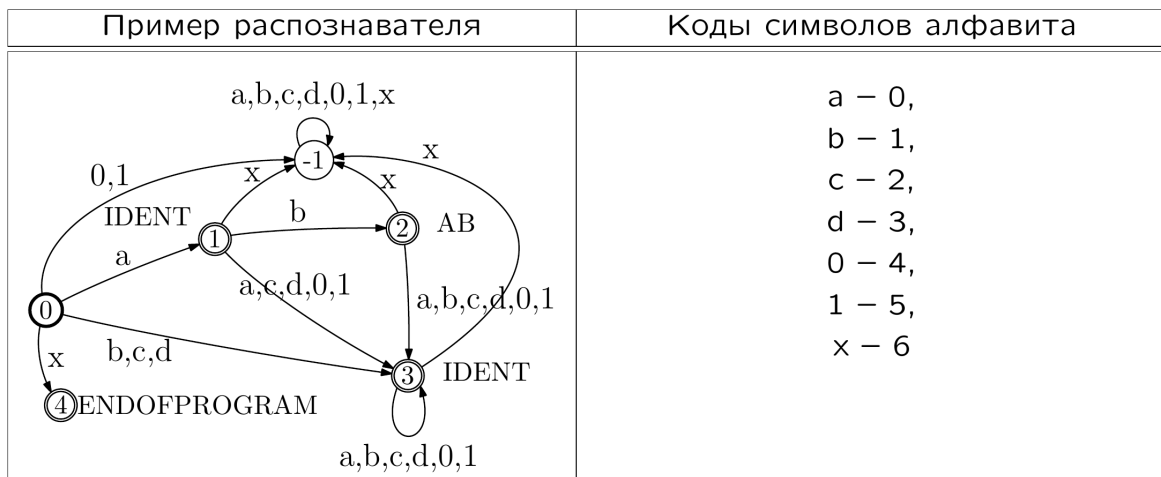
12. Представление лексического распознавателя в программе

Функцию переходов δ детерминированного распознавателя представляют тремя способами:

1. **Таблица переходов** — двумерный массив `table[state, symbol]`, дающий следующее состояние (-1 — нет перехода). Отдельный массив `final[state]` типа `DomainTag[]` хранит разметку (`NOT_FINAL` для незаключительных). Быстрый доступ, но память $O(|Q| \cdot |V|)$.
2. **Списки переходов** — для каждого состояния хранится список пар (символ, целевое состояние). Экономит память на разреженных таблицах.
3. **Алгоритм переходов** — переходы «зашиты» в код (последовательность `if/switch`).

Факторизация алфавита. Символы x и y называются *эквивалентными*, если $\forall q \in Q \delta(q, x) = \delta(q, y)$. Группе эквивалентных символов в таблице переходов соответствует один столбец — это уменьшает таблицу.

Пример. Распознаватель (-1 — состояние-ловушка) и коды символов алфавита:



Его представление через таблицу переходов `int[,]` и массив разметки `DomainTag[]`:

Применение таблицы переходов:

```
private int[,] table = new int[,]
{
    /*
        /* a b c d 0 1 x */
        /* state 0 */ { 1, 3, 3, 3, -1, -1, 4 },
        /* state 1 */ { 3, 2, 3, 3, 3, 3, -1 },
        /* state 2 */ { 3, 3, 3, 3, 3, 3, -1 },
        /* state 3 */ { 3, 3, 3, 3, 3, 3, -1 },
        /* state 4 */ { -1, -1, -1, -1, -1, -1, -1 }
    };

private DomainTag[] final = new DomainTag[]
{
    /* state 0 */ NOT_FINAL,
    /* state 1 */ IDENT,
    /* state 2 */ AB,
    /* state 3 */ IDENT,
    /* state 4 */ ENDOFPROGRAM
};
```

13. Генератор лексических анализаторов flex

flex — вариант генератора lex; по описанию порождает код анализатора на C (файл lex.yy.c) — тело функции yylex().

Структура описания (три секции, разделённые %):

```
Секция определений
Секция правил
Пользовательский код
```

Секция определений: именованные регулярные выражения (Имя РегВыр) и стартовые условия (%x ИмяУсловия). Стартовые условия позволяют переключать анализатор в специальные режимы (распознавание комментариев, строк и т.п.).

Язык регулярных выражений (фрагмент): x — литера; . — любая, кроме перевода строки; [xyz], [a-z], [^A-Z] — классы; r*, r+, r?, r{2,5} — повторения; {Имя} — подстановка; r|s, rs, r/s (опережающий контекст); ^r, r\$; <<E0F>>.

Секция правил: строки вида «регулярное выражение действие». Зависимость от стартовых условий: <n>r, <*>r. Переключение условий — макросом BEGIN(имя); BEGIN(0) возвращает к INITIAL.

Опции: %option nouwrap bison-bridge bison-locations — при «мосте» с bison прототип становится

```
int yylex(YSTYPE *yylval, YYLTYPE *yylloc);
```

Функция возвращает тег токена (0 — конец), атрибут — через yylval, координаты — через yylloc.

Переменные в действиях: yytext (строка-лексема), yyleng (длина), yylval (атрибут, out), yylloc (координаты, out). Общий код начала действий выносят в макрос YY_USER_ACTION (например, для подсчёта координат).

14. Постановка задачи синтаксического анализа

КС-грамматика. (контекстно-свободная) — это $G = \langle N, T, P, S \rangle$, в которой каждое правило из P имеет вид $A \rightarrow \gamma$, где $A \in N$ — нетерминал, $\gamma \in (N \cup T)^*$ — цепочка.

Вывод. $uAv \Rightarrow u\alpha v$ (вывод за один шаг), если $A \rightarrow \alpha \in P$. Запись $u_1 \Rightarrow^* u_n$ — вывод за ноль и более шагов (рефлексивно-транзитивное замыкание), $\Rightarrow^+$ — за один и более.

Сентенциальная форма. — цепочка u такая, что $S \Rightarrow^* u$. *Левая* (правая) сентенциальная форма — выводимая левосторонним (правосторонним) выводом, т.е. подстановкой всякий раз самого левого (правого) нетерминала. *Предложение* — сентенциальная форма без нетерминалов.

Дерево вывода. в КС-грамматике — упорядоченное дерево, вершины которого помечены символами из $N \cup T \cup \{\varepsilon\}$, причём: (1) корень помечен аксиомой S ; (2) листья — терминалами или ε ; (3) внутренние вершины — нетерминалами; (4) если вершина помечена N , а её потомки слева направо — $X_1 \dots X_n$, то $N \rightarrow X_1 \dots X_n$ — правило из P .

Задача синтаксического анализа. Дано: КС-грамматика G и цепочка $u \in T^*$. Требуется определить, является ли u предложением в G , и если да — построить дерево вывода. *Замечание:* реальный анализатор не обязан строить настоящее дерево — достаточно формировать данные (например, последовательность правил), по которым его можно восстановить.

15. Переход от абстрактного синтаксиса к конкретному. Выражение приоритета и ассоциативности операций в грамматике

Абстрактный синтаксис. — упрощённая грамматика, в которой нет информации, гарантирующей единственность дерева вывода.

Пример (арифметические выражения):

$$\text{Expr} ::= \text{Expr BinOp Expr} \mid \text{UnOp Expr} \mid \text{Var} \mid \text{Number}.$$

Такая грамматика *неоднозначна*: для $a+b*c$ строятся два разных дерева.

Конкретный синтаксис устраняет неоднозначность, кодируя приоритет и ассоциативность через иерархию нетерминалов:

$$\begin{aligned} \text{Expr} &::= \text{Expr AddOp Term} \mid \text{Term} \\ \text{Term} &::= \text{Term MulOp Factor} \mid \text{Factor} \\ \text{Factor} &::= "(" \text{Expr} ")" \mid \text{Var} \mid \text{Number} \end{aligned}$$

Приоритет выражается *уровнями нетерминалов*: операции с меньшим приоритетом (+, -) — на верхнем уровне (Expr), с большим (*, /) — на нижнем (Term); скобки и атомы — на самом нижнем (Factor). Чем ниже уровень, тем «крепче связывает» операция.

Ассоциативность выражается *видом рекурсии*: левая рекурсия ($\text{Expr} ::= \text{Expr AddOp Term}$) даёт левую ассоциативность ($a - b - c = (a - b) - c$); правая рекурсия — правую.

16. Алгоритм предсказывающего синтаксического разбора

Предсказывающий (нисходящий, top-down) анализ строит дерево вывода сверху вниз, от аксиомы к листьям. Главная проблема — выбор правила для раскрытия очередного нетерминала.

Структура анализатора: вход (цепочка с маркером \$ в конце), магазин (стек, на дне \$, в начале сверху аксиома S), таблица $\delta: N \times T \rightarrow (N \cup T)^* \cup \{\text{ERROR}\}$, программа.

Работа. Пусть X — верхушка магазина, a — текущий символ входа:

1. если $X = a = \$$ — успех, разбор окончен;
2. если $X = a \neq \$$ — снять X с магазина и продвинуть вход (*сопоставление терминала*);
3. если X — нетерминал, посмотреть $\delta(X, a)$: если там правило $X \rightarrow Y_1 Y_2 \dots Y_k$, то снять X и поместить в магазин $Y_k \dots Y_2 Y_1$ (так, чтобы Y_1 оказался сверху), а правило выдать на выход; если ERROR — перейти к восстановлению при ошибках.

На выходе — последовательность применённых правил (левый вывод). Таблицу δ удаётся построить однозначно только для LL(1)-грамматик (см. вопросы 18, 19, 21).

17. Множества FIRST и FOLLOW

FIRST(u). — множество терминалов, с которых начинаются цепочки, выводимые из u ; если $u \Rightarrow^* \varepsilon$, то $\varepsilon \in \text{FIRST}(u)$:

$$\text{FIRST}(u) = \{a \in T \mid u \Rightarrow^* av\} \cup \{\varepsilon \mid u \Rightarrow^* \varepsilon\}.$$

FOLLOW(X). — множество терминалов, которые могут следовать непосредственно за X в какой-либо сентенциальной форме: $\text{FOLLOW}(X) = \{a \in T \mid S \Rightarrow^* uXav\}$. Если X может быть самым правым символом, то $\$ \in \text{FOLLOW}(X)$.

Функция $F[u]$ вычисляет FIRST для произвольной цепочки по известным FIRST нетерминалов:

$$F[au] = \{a\}, \quad F[Xu] = \begin{cases} \text{FIRST}(X), & \varepsilon \notin \text{FIRST}(X) \\ (\text{FIRST}(X) \setminus \{\varepsilon\}) \cup F[u], & \varepsilon \in \text{FIRST}(X) \end{cases}, \quad F[\varepsilon] = \{\varepsilon\}.$$

Алгоритм построения FIRST (нетерминалов): (1) $\text{FIRST}(X) = \emptyset$; (2) для каждого правила $X \rightarrow u$ добавить $F[u]$ в $\text{FIRST}(X)$; (3) повторять (2) до стабилизации.

Алгоритм построения FOLLOW: (1) $\text{FOLLOW}(X) = \emptyset$; (2) $\$ \in \text{FOLLOW}(S)$; (3) для правила $X \rightarrow uYv$ добавить $\text{FIRST}(v) \setminus \{\varepsilon\}$ в $\text{FOLLOW}(Y)$; (4) для правил $X \rightarrow uY$ или $X \rightarrow uYv$, где $v \Rightarrow^* \varepsilon$, добавить $\text{FOLLOW}(X)$ в $\text{FOLLOW}(Y)$; повторять до стабилизации.

Пример (грамматика $E \rightarrow TE', E' \rightarrow +TE' \mid \varepsilon, T \rightarrow FT', T' \rightarrow *FT' \mid \varepsilon, F \rightarrow n \mid (E)$):

$$\text{FIRST}(E) = \text{FIRST}(T) = \text{FIRST}(F) = \{ (, n \}$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}, \quad \text{FIRST}(T') = \{ *, \varepsilon \}$$

$$\text{FOLLOW}(E) = \text{FOLLOW}(E') = \{), \$ \}$$

$$\text{FOLLOW}(T) = \text{FOLLOW}(T') = \{ +,), \$ \}$$

$$\text{FOLLOW}(F) = \{ +, *,), \$ \}$$

18. LL(1)-грамматики

LL(1)-грамматика. — КС-грамматика, в которой для любых двух правил $A \rightarrow u \mid v$ выполнено: (1) $\text{FIRST}(u) \cap \text{FIRST}(v) = \emptyset$; (2) если $v \Rightarrow^* \varepsilon$, то $\text{FIRST}(u) \cap \text{FOLLOW}(A) = \emptyset$.

Эти условия гарантируют, что по вершущке магазина A и одному входному символу всегда однозначно определяется правило:

- если $a \in \text{FIRST}(u) \cap \text{FIRST}(v)$ — неясно, каким правилом раскрывать A ;
- если $\varepsilon \in \text{FIRST}(u) \cap \text{FIRST}(v)$ и $a \in \text{FOLLOW}(A)$ — неясно, каким из правил «свернуться в ε »;
- если $a \in \text{FIRST}(u) \cap \text{FOLLOW}(A)$ — неясно, раскрыть A как u или как ε .

Однозначную таблицу предсказывающего разбора можно построить тогда и только тогда, когда грамматика — LL(1).

Грамматики языков часто «почти LL(1)» — мешают *левая рекурсия* и необходимость *левой факторизации*.

Удаление левой рекурсии (простейший случай). Правила $A \rightarrow Av_1 \mid \dots \mid Av_n \mid u_1 \mid \dots \mid u_m$ преобразуют в:

$$A \rightarrow u_1 A' \mid \dots \mid u_m A', \quad A' \rightarrow v_1 A' \mid \dots \mid v_n A' \mid \varepsilon.$$

Левая факторизация. Если есть $A \rightarrow uv_1 \mid uv_2$ (общий префикс u), решение откладывают:

$$A \rightarrow uA', \quad A' \rightarrow v_1 \mid v_2.$$

19. Построение таблиц предсказывающего анализатора

Алгоритм работает для любой КС-грамматики, строя $\delta': N \times T \rightarrow 2^{(N \cup T)^*} \cup \{\text{ERROR}\}$ (в ячейке в общем случае может быть несколько правил — наличие такой ячейки и означает, что грамматика не LL(1)).

Алгоритм. Изначально $\delta'(X, a) = \text{ERROR}$ для всех X, a . Для каждого правила $X \rightarrow u$:

- для всех $a \in \text{FIRST}(u)$ добавить u в $\delta'(X, a)$;
- если $\varepsilon \in \text{FIRST}(u)$, то для всех $b \in \text{FOLLOW}(X)$ добавить u в $\delta'(X, b)$.

«Добавить» означает: если было ERROR, ячейка становится $\{u\}$; иначе u добавляется к уже имеющемуся множеству.

Пример — итоговая таблица (грамматика из вопроса 17):

	+	*	n	(	)	\$
E			TE'	TE'		
E'	$+TE'$				ε	ε
T			FT'	FT'		
T'	ε	$*FT'$			ε	ε
F			n	(E)		

(Пустые ячейки соответствуют ERROR.) Поскольку нигде не возникло двух правил в одной ячейке — грамматика LL(1).

20. Представление дерева вывода в программе. Построение дерева вывода в предсказывающем синтаксическом анализе

Представление дерева в программе. Дерево вывода удобно представлять объектами-узлами: каждый узел хранит метку (нетерминал или терминал) и упорядоченный список ссылок на узлы-потомки. Листья соответствуют терминалам и ε . Часто узлы — это объекты классов, наследующих от общего абстрактного класса узла (по узлу на нетерминал / альтернативу). На практике строят не абстрактное дерево разбора в чистом виде, а абстрактное синтаксическое дерево (AST), отбрасывая лишние промежуточные узлы.

Построение в предсказывающем анализе. Базовый алгоритм (вопрос 16) выдаёт лишь последовательность применённых правил — левый вывод. Чтобы строить дерево, его дополняют: вместе с символом в магазин кладут ссылку на соответствующий ему узел дерева.

- Вначале в магазин помещают аксиому S вместе с созданным корневым узлом.
- При раскрытии нетерминала X по правилу $X \rightarrow Y_1 \dots Y_k$ для узла X создают k дочерних узлов (по одному на Y_i), связывают их с X как упорядоченных потомков и помещают в магазин символы $Y_k \dots Y_1$ вместе со ссылками на эти узлы.
- При сопоставлении терминала соответствующий лист дерева получает атрибут токена (значение лексемы, координаты).

Поскольку предсказывающий разбор порождает *левый* вывод, дерево достраивается сверху вниз и слева направо.

21. Рекурсивный спуск

Метод рекурсивного спуска применяется к LL(1)-грамматикам, записанным в РБНФ (см. вопрос 23). Каждому нетерминалу сопоставляется отдельная *функция*, кодирующая применение его (единственного) правила.

Соглашения для функции нетерминала X :

- существует глобальная переменная `Sum`, в которую до вызова функции помещён токен первого символа раскрываемой цепочки;
- функция должна либо полностью потребить цепочку, в которую раскрывается X , либо породить сообщение об ошибке;
- после возврата `Sum` содержит токен, непосредственно следующий за раскрытым X .

Шаблоны порождения тела `PARSE(R)` по структуре правила РБНФ:

Конструкция R	Код
ε	; (ничего)
терминал a	if (Sym==a) Sym=NextToken(); else ReportError();
нетерминал X	X();
$R_1 R_2$	PARSE(R1); PARSE(R2);
$R_1 \mid R_2$	if (Sym∈FIRST(R1)) PARSE(R1); else PARSE(R2);
R^*	while (Sym∈FIRST(R)) PARSE(R);
R^+	do PARSE(R); while (Sym∈FIRST(R));
$R^?$	if (Sym∈FIRST(R)) PARSE(R);

Пример. Из правила $\text{Expr} ::= ('+' | '-')? \text{Term} (('+' | '-') \text{Term})^*$ после последовательного раскрытия и упрощения получается:

```
void Expr()
if (Sym=='+' || Sym=='-') Sym=NextToken();
Term();
while (Sym=='+' || Sym=='-')
Sym=NextToken();
Term();
```

Аналогично порождаются `Term()` и `Factor()`. Достоинства метода — простота и прозрачность; код анализатора напоминает саму грамматику.

22. Алгоритм Эрли

Алгоритм Эрли (Earley, 1968) подходит для *любой* КС-грамматики и для заданного предложения находит все левые выводы. Это нисходящий алгоритм. Оценка времени: $O(n)$ для LL(1)-грамматик, $O(n^2)$ для однозначных, $O(n^3)$ для остальных.

Работа основана на построении *множеств Эрли*. Для цепочки $a_1 a_2 \dots a_n$ строится $n + 1$ множеств: S_0 и по одному S_i на каждый входной символ.

Элемент множества Эрли. (в S_k) — тройка $\langle p, i, j \rangle$: p — правило; i — позиция точки $\bullet$ в правой части (сколько уже обработано); $j \leq k$ — номер множества, в котором началось распознавание по p . Записывается как $[A \rightarrow u \bullet v, j]$.

В S_0 кладётся стартовый элемент $[S' \rightarrow \bullet S, 0]$, где $S' \rightarrow S$ — добавленное вспомогательное правило. Для восстановления вывода строится *дерево потомков* (один элемент порождает другой).

Три операции (на шаге i , A, B — нетерминалы, b — терминал):

- **SCANNER**(x): если $x = [A \rightarrow \dots \bullet b \dots, j]$ и $b = a_{i+1}$, добавить $[A \rightarrow \dots b \bullet \dots, j]$ в S_{i+1} .
- **PREDICTOR**(x): если $x = [A \rightarrow \dots \bullet B \dots, j]$, то для каждого правила $B \rightarrow u$ добавить $[B \rightarrow \bullet u, i]$; если $B \Rightarrow^* \varepsilon$, добавить также $[A \rightarrow \dots B \bullet \dots, j]$.
- **COMPLETER**(x): если $x = [A \rightarrow \dots \bullet, j]$ (точка в конце), то для каждого $[B \rightarrow \dots \bullet A \dots, k] \in S_j$ добавить $[B \rightarrow \dots A \bullet \dots, k]$.

Предложение принадлежит языку, если в S_n есть элемент $[S' \rightarrow S \bullet, 0]$. SCANNER «продвигает точку» по терминалу, PREDICTOR «предсказывает» раскрытие нетерминала, COMPLETER «завершает» распознанный нетерминал и продвигает зависящие от него элементы.

23. Расширенная БНФ. Построение множества FIRST для РБНФ

РБНФ. (расширенная форма Бэкуса-Наура) — запись КС-грамматики, в которой правила имеют вид $X \rightarrow R$, где R — дерево, синтаксис которого задаётся:

$$R ::= \varepsilon \mid a \mid X \mid RR \mid R \mid R^* \mid R^+ \mid R?$$

(ε — пустое; a — терминал; X — нетерминал; конкатенация; альтернатива; повторения $0+$, $1+$, $0/1$).

Приоритет операций (по убыванию): $*$, $+$, $?$; затем конкатенация; затем альтернатива. Скобки меняют приоритет. *Нотация Вирта* — альтернативная запись: $=$ (определение), $.$ (конец), $|$, $[]$ ($0/1$), $\{ \}$ ($0+$), $()$ (приоритет).

FIRST для выражений РБНФ. Функция $F: \text{RHS} \rightarrow 2^T$:

$$\begin{aligned} F[a] &= \{a\}, & F[X] &= \text{FIRST}(X), & F[\varepsilon] &= \{\varepsilon\}, \\ F[uv] &= \begin{cases} F[u], & \varepsilon \notin F[u] \\ (F[u] \setminus \{\varepsilon\}) \cup F[v], & \varepsilon \in F[u] \end{cases}, & F[u \mid v] &= F[u] \cup F[v], \\ F[u?] &= F[u] \cup \{\varepsilon\}, & F[u^*] &= F[u] \cup \{\varepsilon\}, & F[u^+] &= F[u]. \end{aligned}$$

Алгоритм для нетерминалов тот же, что и в БНФ: инициализировать пустыми, для каждого правила $X \rightarrow u$ добавлять $F[u]$ в $\text{FIRST}(X)$, повторять до стабилизации.

24. Представление одноместных операций и ассоциативности двуместных операций в LL(1) РБНФ

Двуместные (бинарные) операции и ассоциативность. В LL(1)-РБНФ левая рекурсия запрещена, поэтому повторение кодируют итерацией $*$:

$$\text{Expr} ::= \text{Term} (('+' \mid '-') \text{Term})^*.$$

Дерево разбора, порождаемое циклом `while`, по структуре правоассоциативно, поэтому *левую ассоциативность* реализуют не в грамматике, а в коде/семантике: накапливают результат в цикле слева направо (после разбора очередного `Term` «сворачивают» его с уже накопленным левым операндом). Так из $a-b-c$ получают $(a-b)-c$.

Одноместные (унарные) операции. Префиксный унарный оператор кодируют необязательным префиксом или повторением:

$$\text{Expr} ::= ('+' \mid '-')? \text{Term} \dots$$

Для нескольких подряд идущих префиксных операций (например, несколько унарных минусов) используют $('-')^*$. Поскольку унарный минус правоассоциативен ($-(-a) = -(-a)$), рекурсивный спуск естественно обрабатывает его правильно: рекурсивный/итеративный разбор префиксов сначала «снимает» внешние операторы.

25. Синтаксический разбор типа «перенос-свёртка». Построение дерева вывода при этом разборе

ПС-разбор (shift-reduce) — восходящий (bottom-up): строит *правосторонний* вывод предложения в *обратном* порядке (от цепочки к аксиоме).

Основа. (handle) правой сентенциальной формы u — сочетание правила $X \rightarrow v$ и позиции вхождения v в u , такое, что замена этого v на X даёт предыдущую правую сентенциальную форму. После основы в форме идут только терминалы.

Структуры данных: стек символов грамматики (на дне $\$$) и входной буфер (нераспознанный суффикс, оканчивается $\$$). Начало: стек $\$$, буфер $u\$$.

Четыре действия:

1. **перенос** (shift) — символ из начала буфера переносится в стек;
2. **свёртка** (reduce) — основа на вершине стека заменяется нетерминалом;
3. **допуск** (accept) — успех: в буфере $\$$, в стеке $\$S$;
4. **ошибка**.

Основные проблемы — поиск основы в стеке и выбор правила свёртки (в общем случае возможны возвраты; их устраняют SLR/LR-распознаватели, вопросы 26–29).

Пример разбора $n^*(n+n)$ для $E \rightarrow E+T \mid T, T \rightarrow T*F \mid F, F \rightarrow n \mid (E)$ — последовательность переносов и свёрток ($F \rightarrow n, T \rightarrow F, \dots$), завершающаяся допуском в стеке $\$E$.

Построение дерева. При каждой свёртке по $X \rightarrow v$ создаётся узел X , потомками которого становятся узлы $|v|$ символов основы, снимаемых со стека (их ссылки также хранятся в стеке рядом с символами). Так дерево растёт снизу вверх; корень S создаётся при заключительной свёртке. Порядок свёрток обратен правому выводу.

26. Недетерминированные SLR-распознаватели

Распознаватели активных префиксов помогают решить проблемы ПС-алгоритма (поиск основы и выбор правила свёртки).

Активный префикс. (viable prefix) — префикс правосентенциальной формы, не выходящий за правый конец её основы. Если ПС-алгоритм работает правильно, в его стеке выше $\$$ лежит активный префикс.

SLR-ситуация. (item) — пара $\langle p, i \rangle$: правило p и позиция точки i . Записывается $[X \rightarrow u \bullet v]$. Для правила длины n существует $n + 1$ ситуаций; для $X \rightarrow \varepsilon$ — единственная $X \rightarrow \bullet$.

Расширенная грамматика. G' для $G = \langle N, T, P, S \rangle$ — это $G' = \langle N \cup \{S'\}, T, P \cup \{S' \rightarrow S\}, S' \rangle$ (добавлен новый стартовый символ и правило).

Недетерминированный SLR-распознаватель. активных префиксов — конечный автомат, состояния которого — все SLR-ситуации для P' , а переходы:

- 1) $[X \rightarrow u \bullet xv] \xrightarrow{x} [X \rightarrow ux \bullet v]$, где $x \in N' \cup T$ (сдвиг точки по символу);
- 2) $[X \rightarrow u \bullet Yv] \xrightarrow{\varepsilon} [Y \rightarrow \bullet w]$, где $Y \in N'$ (ε -переход — «предсказание» раскрытия Y).

Начальное состояние — $[S' \rightarrow \bullet S]$; заключительные — все ситуации вида $[X \rightarrow u \bullet]$.

Использование. «Натравив» распознаватель на активный префикс из стека, получают

подсказку для РС-алгоритма:

- заключительная ситуация $[X \rightarrow u\bullet]$ — нужна *свёртка* по $X \rightarrow u$;
- ситуация $[X \rightarrow u\bullet xv]$ — нужен *перенос*;
- распознавание невозможно — обнаружена *ошибка*;
- получено более одной ситуации — грамматика слишком сложна для SLR (*конфликт*).

Бывают конфликты «перенос/свёртка» и «свёртка/свёртка».

27. Детерминированные SLR-распознаватели

Если детерминизировать недетерминированный SLR-распознаватель, получится *детерминированный* SLR-распознаватель, состояниями которого являются не отдельные ситуации, а *множества ситуаций* (как и при детерминизации лексического распознавателя — subset construction с ε -замыканием).

Начальное состояние содержит ситуацию $[S' \rightarrow \bullet S]$ (вместе с её ε -замыканием); заключительные состояния — те, что содержат заключительные ситуации недетерминированного распознавателя.

На практике используют только детерминированные SLR-распознаватели: они распознают активный префикс за время, пропорциональное его длине.

Каждое состояние получают так: к ядру (множеству ситуаций) добавляют замыкание — для каждой ситуации $[X \rightarrow u\bullet Yv]$ дописывают все $[Y \rightarrow \bullet w]$. Переход по символу x ведёт в состояние, образованное сдвигом точки во всех подходящих ситуациях с последующим замыканием. Если в одном состоянии оказываются и заключительная ситуация, и ситуация с точкой перед терминалом, имеет место конфликт «перенос/свёртка» (пример — состояние с $[S \rightarrow L\bullet = R]$ и $[R \rightarrow L\bullet]$).

28. Алгоритм LR(1)-анализа

LR(1)-анализ — вариант РС-алгоритма, работающий за линейное время и управляемый *бесконфликтными* таблицами распознавателя. В стеке хранится последовательность *состояний* распознавателя, пройденных при распознавании активного префикса.

Две таблицы:

- $action: State \times T \rightarrow \{\text{перенос } s', \text{ свёртка } X \rightarrow u, \text{ допуск, ошибка}\};$
- $goto: State \times N' \rightarrow State \cup \{\text{ошибка}\}.$

Алгоритм (стек начинается с $\$$ и s_0 , a — текущий символ):

- если $action[s, a] = \text{«перенос } s'\text{»}$: поместить s' в стек, прочесть следующий символ;
- если $action[s, a] = \text{«свёртка } X \rightarrow u\text{»}$: снять $|u|$ состояний; пусть s' — новая вершина; поместить $goto[s', X]$; выдать правило $X \rightarrow u$ в результат;
- если «допуск» — вернуть результат; иначе — ошибка.

Заполнение таблиц (для SLR(1)):

- $action[s, a] \ni \text{«перенос } s'\text{»}$, если $s \xrightarrow{a} s'$;
- $action[s, a] \ni \text{«свёртка } X \rightarrow u\text{»}$, если $[X \rightarrow u\bullet] \in s$ и $a \in FOLLOW(X)$;
- $action[s, \$] \ni \text{«допуск»}$, если $[S' \rightarrow S\bullet] \in s$;
- иначе — «ошибка»;

- $goto[s, X] = s'$, если $s \xrightarrow{X} s'$, иначе ошибка.

Использование FOLLOW(X) при свёртках — характерная черта именно SLR(1); более мощные LR(1)/LALR(1) используют точечные lookahead-символы, прикреплённые к ситуациям, что снимает часть конфликтов.

29. Синтаксически управляемые определения. Порядок выполнения семантических правил. Восходящее выполнение S-атрибутных определений

Аннотированное дерево разбора. — дерево разбора, с каждым узлом которого связана структура именованных полей — *атрибутов* (числа, строки, поддеревья и т.п.).

Синтаксически управляемое определение. (SDD) — тройка $\langle G, A, R \rangle$: КС-грамматика G ; множество атрибутов A (с каждым символом связано 0 или более); множество семантических правил R (с каждым правилом грамматики связано 0 или более). Семантическое правило для $X \rightarrow x_1 \dots x_n$ имеет вид $b := f(c_1, \dots, c_k)$, где b — атрибут одного из символов правила, c_i — атрибуты символов правила, f — вычисляющая функция.

Атрибутная грамматика. — SDD, в котором функции семантических правил не имеют побочных эффектов. Синтаксически управляемая трансляция — перевод предложений путём вычисления атрибутов.

Синтезируемый атрибут. нетерминала вычисляется по синтезируемым атрибутам дочерних узлов (для терминала — на этапе лексического анализа).

Наследуемый атрибут. вычисляется по атрибутам *родителя или соседних* узлов (с тем же родителем).

Пример: в правилах объявления $D \rightarrow TL$, $L \rightarrow L, v$, $T \rightarrow \text{int}$ атрибут T . Type синтезируемый, а L . In, v . Type — наследуемые.

Порядок выполнения семантических правил. SDD компактно, потому что не задаёт порядок вычислений. Два метода его выбора:

1. **По графу зависимостей атрибутов.** Граф зависимостей — направленный граф, узлы — атрибуты узлов дерева, дуга идёт из c в b , если b зависит от c . Если SDD без ошибок, граф ациклический для любого дерева; любая его *топологическая сортировка* даёт корректный порядок вычислений.
2. **По схеме трансляции.**

Семантическое действие. — семантическое правило, привязанное к определённой позиции в правой части правила (разбивает её на префикс и суффикс).

Действие выполняется в момент, когда распознан префикс.

Схема трансляции. — SDD из семантических действий, удовлетворяющих условиям: (1) наследуемый атрибут символа x вычисляется в действии *перед* x ; (2) действие не обращается к синтезируемому атрибуту символа *справа* от себя; (3) синтезируемый атрибут левой части вычисляется в действии *после* последнего символа правила.

Восходящее выполнение S-атрибутных определений.

S-атрибутное определение. — SDD, в котором применяются только *синтезируемые* атрибуты.

Синтезируемые атрибуты вычисляются восходящим (ПС/LR) анализатором прямо в процессе разбора. К символам грамматики (или состояниям) в стеке добавляют значения синтезируемых атрибутов (можно завести отдельный *стек атрибутов*). При свёртке по $X \rightarrow u$ со стека снимается $|u|$ символов, и по их атрибутам вычисляется синтезируемый атрибут X , который кладётся обратно. *Пример:* при разборе $3*(4+5)$ в стеке атрибутов накапливаются промежуточные значения, и в конце вычисляется 27.

30. Постановка задачи семантического анализа. Области и локальные таблицы символов

Символ. (symbol) — именованная сущность программы, пара $\langle name, info \rangle$ (идентификатор и описание). Сущности: переменные, типы, функции, модули. Один идентификатор могут иметь разные символы $\langle name, info_1 \rangle, \langle name, info_2 \rangle$; реже одна сущность имеет несколько идентификаторов (псевдонимы).

Видимость и время жизни. Символ *виден* в узле дерева, если его идентификатор может там использоваться. Видимость — обычно *статическая* характеристика (определяется при компиляции). Для символов с памятью времени выполнения вводят *время жизни* (lifetime) — *динамическую* характеристику (от выделения до освобождения памяти; в рекурсии символ может присутствовать в нескольких экземплярах).

Таблица символов. для узла — отображение идентификаторов видимых символов в их описания. Реализуется хеш-таблицей; ключи — коды идентификаторов, присвоенные лексическим анализатором (ускоряет хеширование).

Иногда множества идентификаторов разных категорий пересекаются (в С тег структуры может совпадать с именем переменной) — тогда для каждой категории заводят отдельную таблицу.

Две задачи семантического анализа:

1. построить таблицу символов для каждой области и выдать ошибки в узлах, где используются неописанные идентификаторы;
2. проверить допустимость операций над сущностями (для статически типизированных языков — вычислить типы всех узлов-выражений).

Области и локальные таблицы.

Область. (scope) — множество узлов дерева, в которых видны одни и те же символы. Для всей области строится одна таблица символов — это повышает

эффективность.

Вложенная область. наследует таблицу символов родительской области, дополняя её собственными символами; собственные символы могут *экранировать* унаследованные.

Открытость. : область A открыта для B , если B непосредственно вложена в A или специальная конструкция открывает A для B (например, `with` в Pascal, доступ к полям структуры).

Классификация символов области и формула эффективных символов:

$$E = (P \cup I) \setminus S,$$

где P — собственные, I — унаследованные (эффективные символы открытых областей), S — экранированные (унаследованные, чьи идентификаторы совпадают с собственными).

Локальная таблица символов. перечисляет только *собственные* символы области и содержит ссылки на локальные таблицы открытых областей.

Локальные таблицы экономят память (символ входит лишь в одну таблицу). Результат анализа — ациклический ориентированный граф локальных таблиц (корни — самые внешние области). Поиск символа для узла начинается с локальной таблицы его области и при неудаче рекурсивно продолжается в таблицах открытых областей.

31. Проектирование объектно-ориентированных таблиц символов

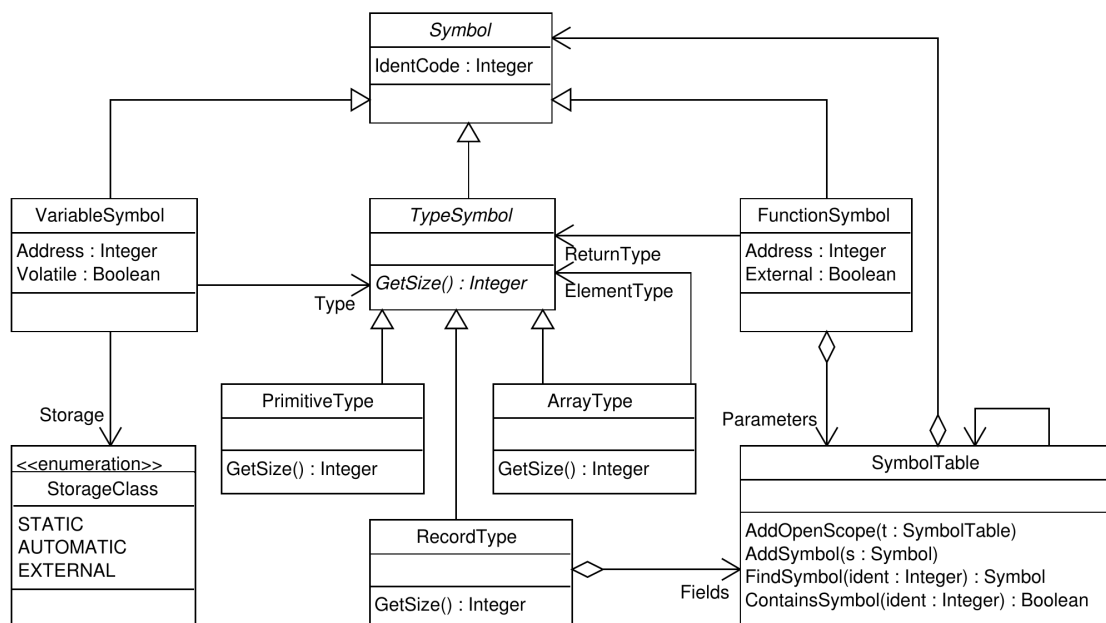
Иерархия классов символов:

- **Symbol** — абстрактный базовый класс; поле `IdentCode : Integer` (код идентификатора от лексического анализатора). Наследники — категории символов: `VariableSymbol`, `TypeSymbol`, `FunctionSymbol`.
- **SymbolTable** — локальная таблица символов. Содержит хеш-таблицу собственных символов и ссылки на таблицы открытых областей. Методы:
 - `AddOpenScope(t : SymbolTable)` — добавить открытую область;
 - `AddSymbol(s : Symbol)` — добавить собственный символ;
 - `FindSymbol(ident) : Symbol` — найти *эффективный* символ (с рекурсией по открытым областям);
 - `ContainsSymbol(ident) : Boolean` — проверить наличие *собственного* символа.
- **VariableSymbol** — переменная. Поля: `Address : Integer` (заполняется на фазе распределения памяти), `Volatile : Boolean`, `Storage` (перечисление `StorageClass`: `STATIC`, `AUTOMATIC`, `EXTERNAL`), `Type : TypeSymbol`.
- **TypeSymbol** — абстрактный класс типа; от него наследуют классы примитивных и составных типов.

Такая объектная модель напрямую отражает понятия из вопроса 30: разделение собственных/эффективных символов реализуется методами `ContainsSymbol/FindSymbol`, а граф локальных таблиц — ссылками `AddOpenScope`.

Сводная диаграмма классов. `Symbol` — абстрактный базовый класс; `VariableSymbol` ссылается на `StorageClass` (классы памяти) и на `TypeSymbol`; от `TypeSymbol` наследуют `PrimitiveType`, `ArrayType`, `RecordType`; `FunctionSymbol` и `RecordType` агрегируют `SymbolTable`

(параметры/поля), а сама `SymbolTable` ссылается на таблицы открытых областей (петля справа):



Методы `SymbolTable`:

Метод	Назначение	Область поиска
<code>AddOpenScope</code>	добавить открытую область	—
<code>AddSymbol</code>	добавить собственный символ	—
<code>FindSymbol</code>	найти эффективный символ	собств. + открытые (рекурсивно)
<code>ContainsSymbol</code>	проверить наличие символа	только собственные

Классы памяти (`StorageClass`) для `VariableSymbol`:

Значение	Размещение	Время жизни
STATIC	статическая память	всё выполнение программы
AUTOMATIC	стек кадра вызова	пока активна функция
EXTERNAL	во внешнем модуле	определяется при компоновке